Chapter 4

Queues

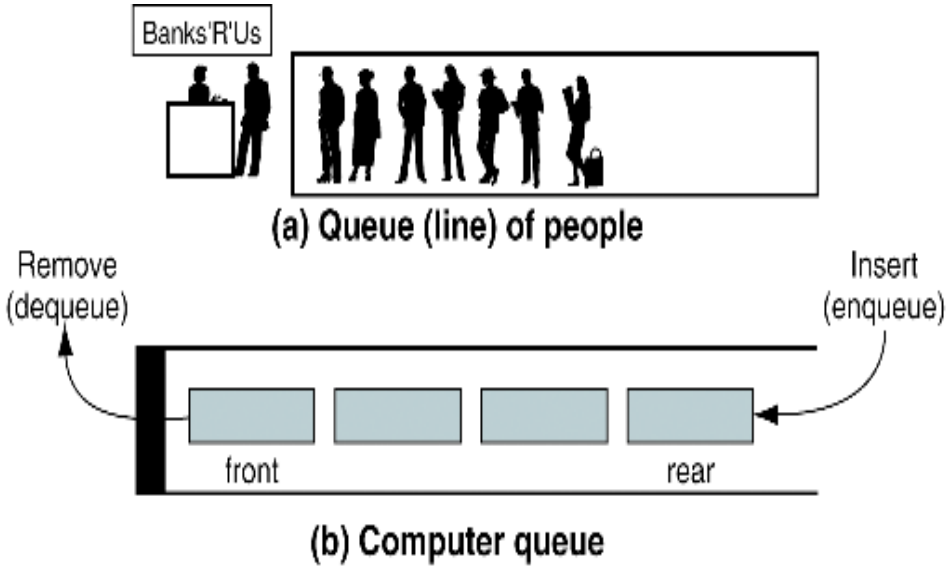


FIGURE 4-1 Queue Concept

- A queue is a linear list in which data can only be inserted at one end (rear), and deleted from the other end (front).
- These restrictions ensure that the data are processed through the queue in the order in which they are received.
- A queue is a first in–first out (FIFO) structure.

Queue Operations

- Enqueue
- Dequeue
- Queue Front
- Queue Rear

Enqueue: put a new element in to the rear of the queue

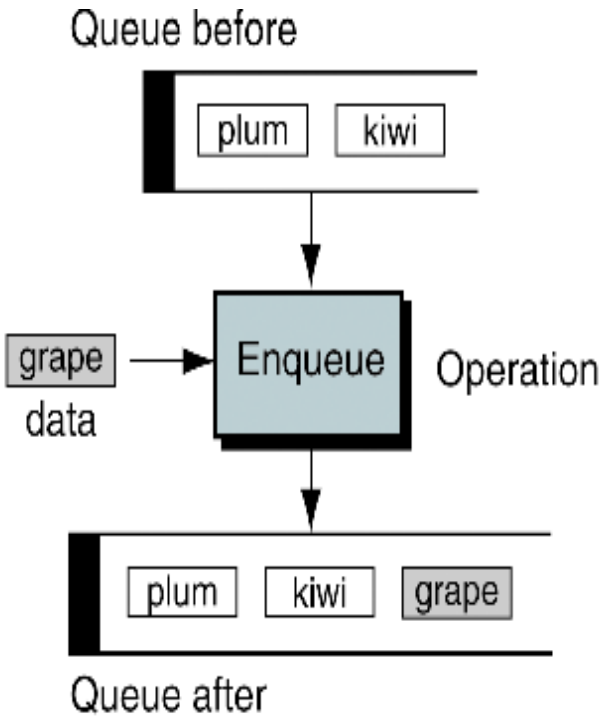


FIGURE 4-2 Enqueue

Dequeue: remove the first element from the front of the queue

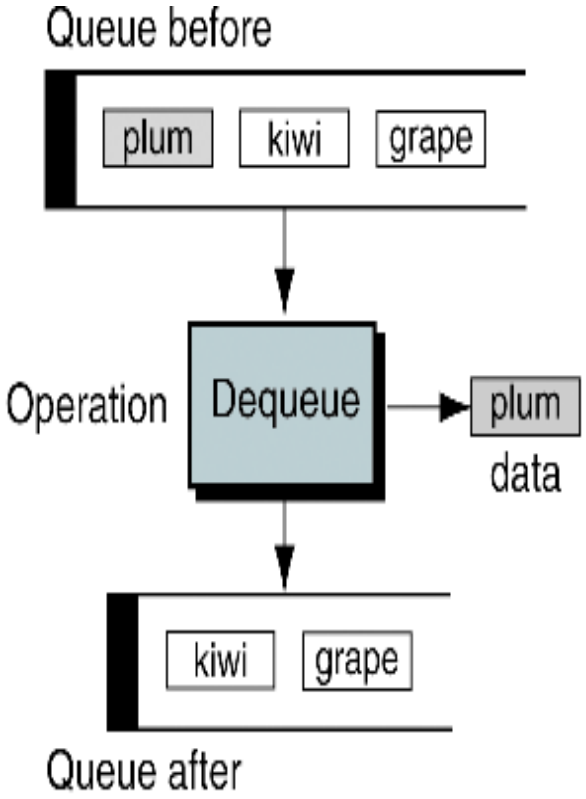


FIGURE 4-3 Dequeue

Queue Front: retrieve the front element

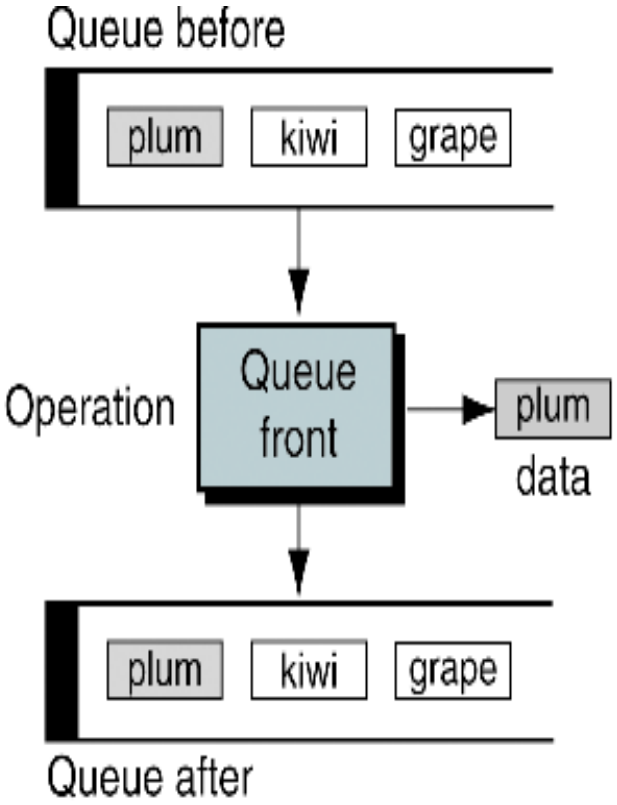


FIGURE 4-4 Queue Front

Queue Rear: retrieve the rear element

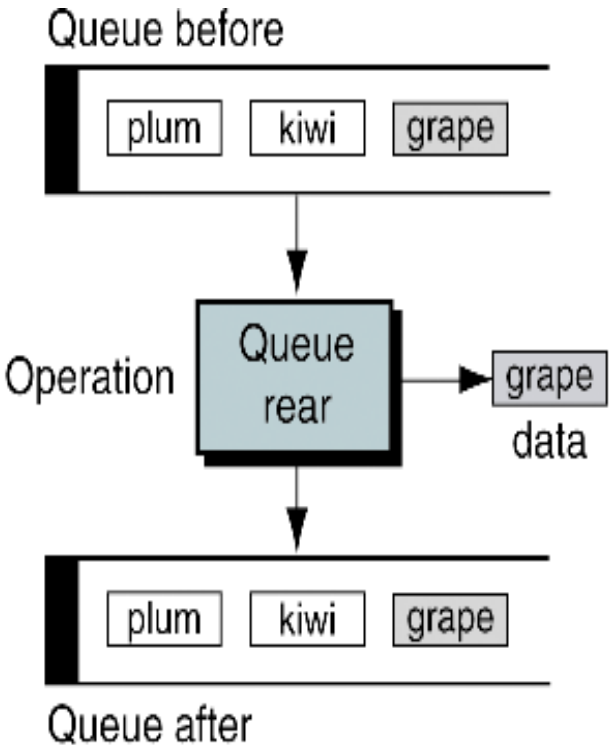


FIGURE 4-5 Queue Rear

FIGURE 4-6 Queue Example

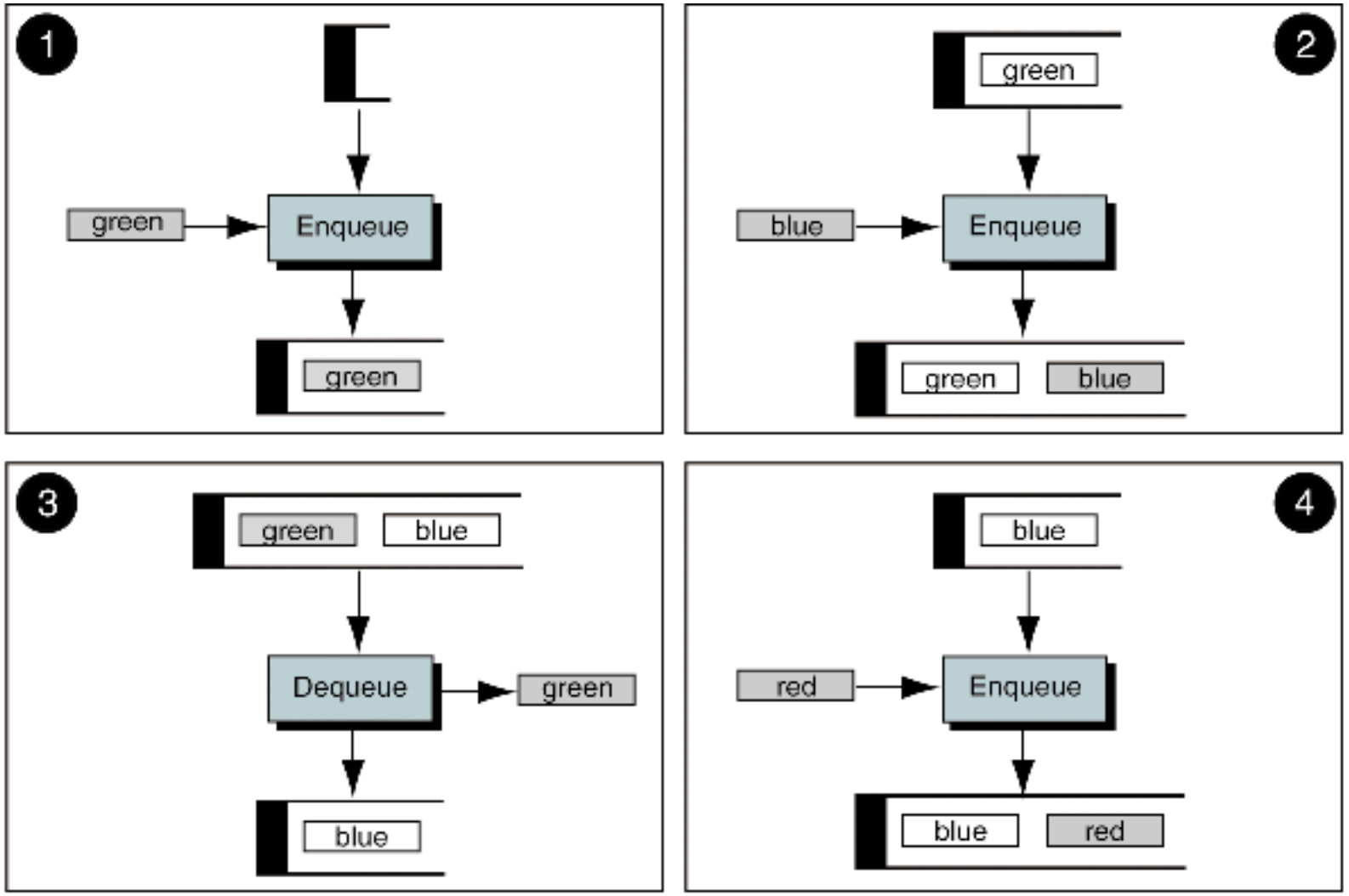
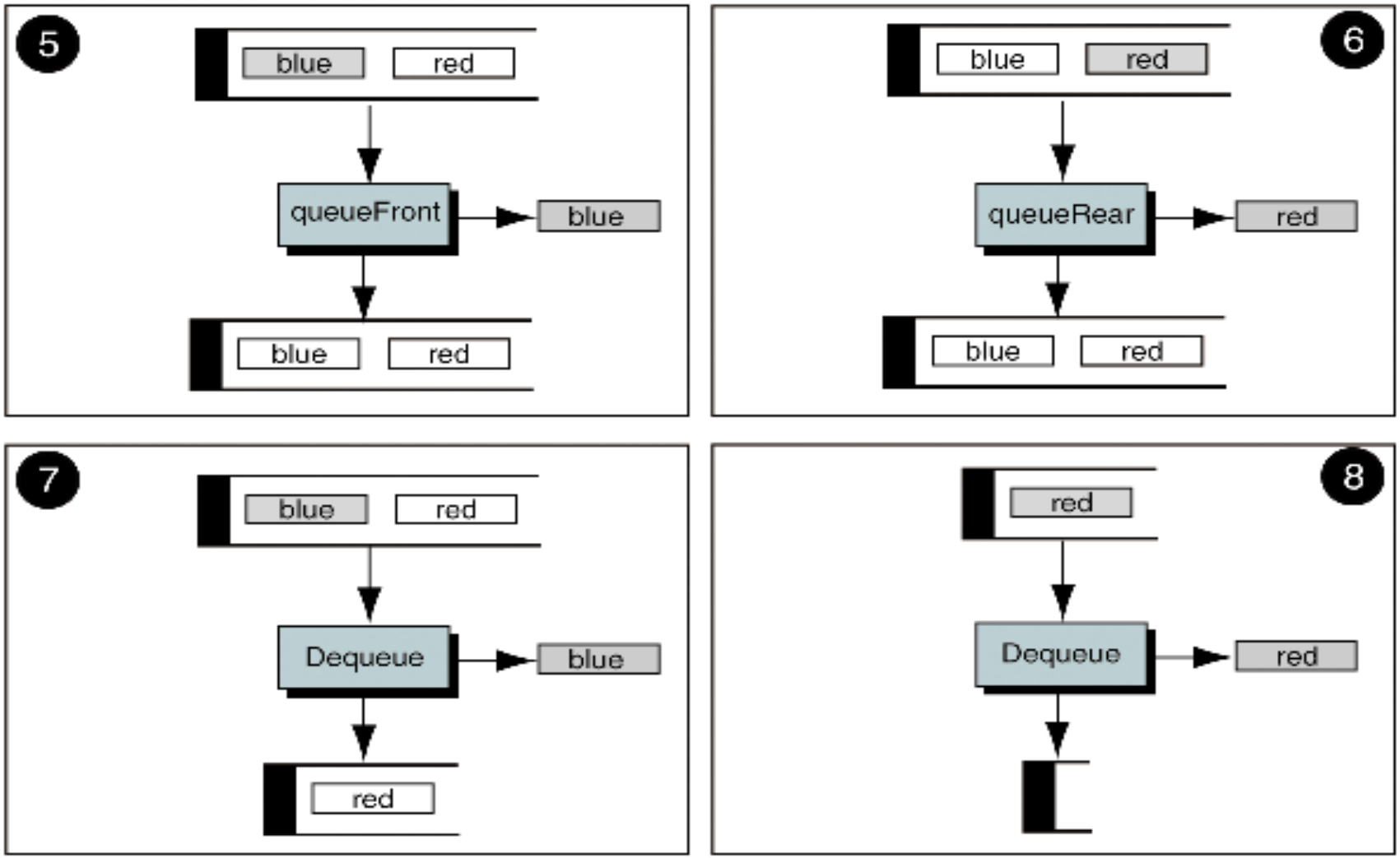
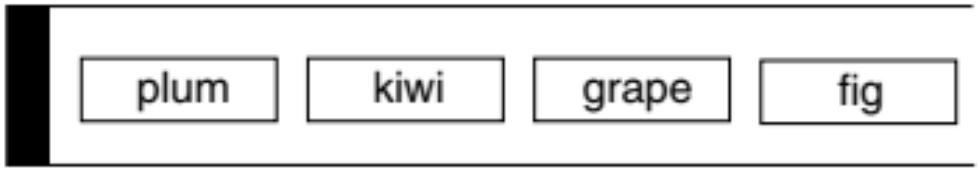


FIGURE 4-6 Queue Example (Continued)

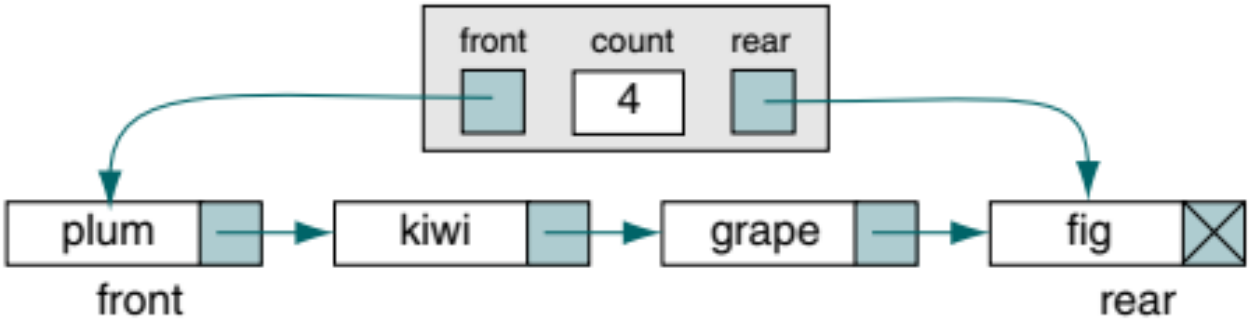


Queue Linked List Design

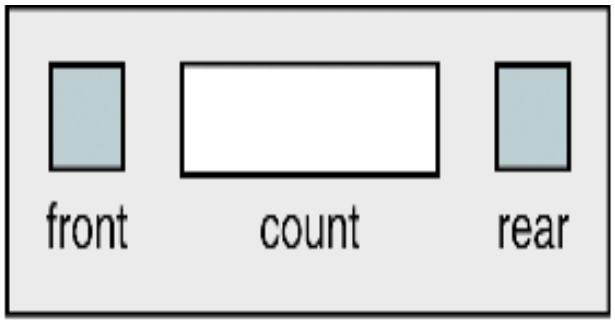
Conceptual and Physical Queue Implementations



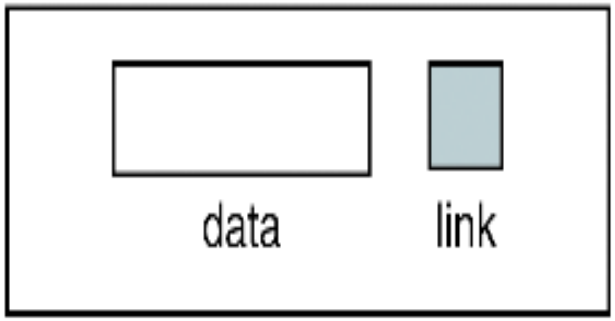
(a) Conceptual queue



(b) Physical queue



Head structure



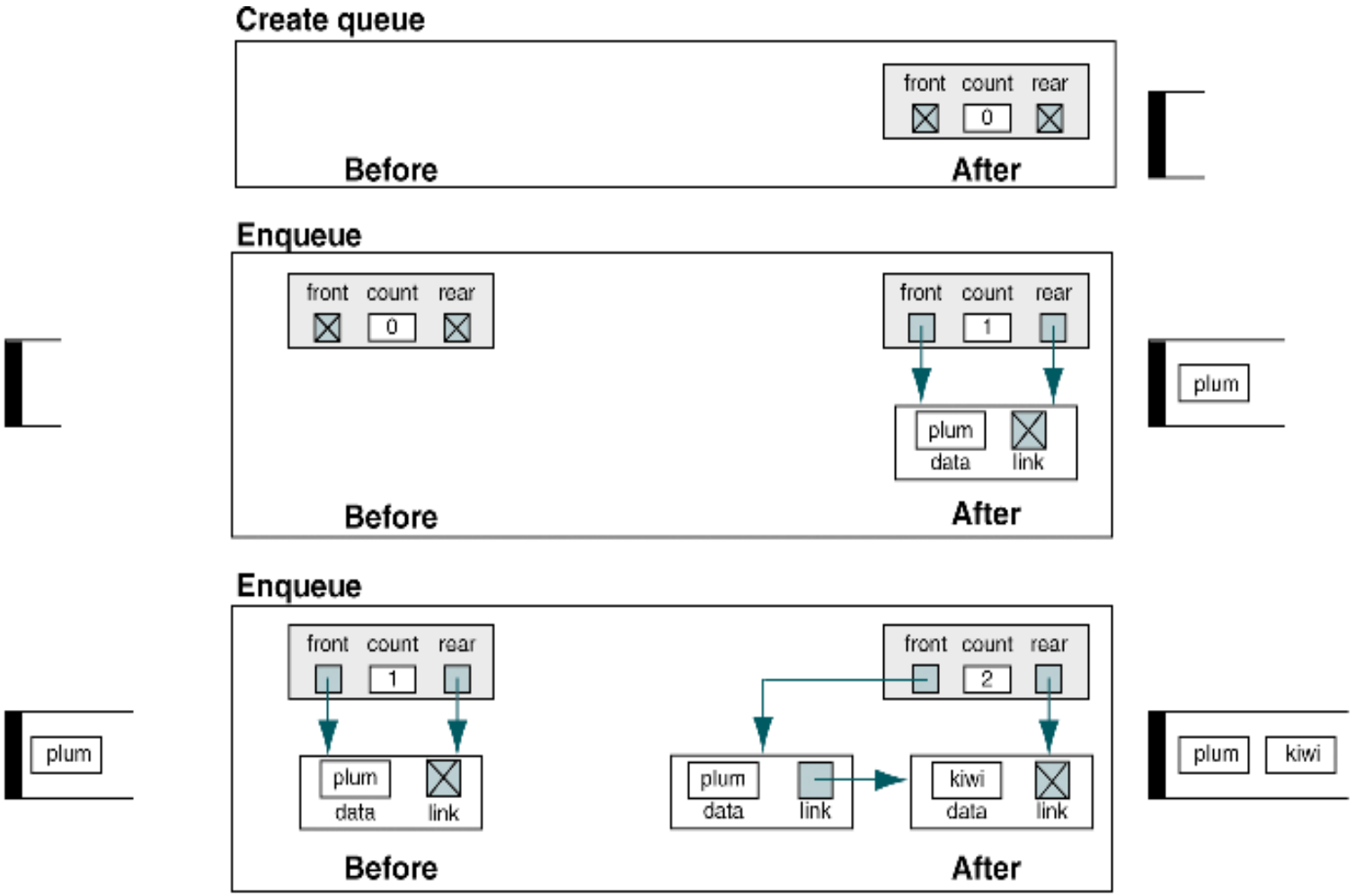
Node structure

```
queueHead
  front
  count
  rear
end queueHead

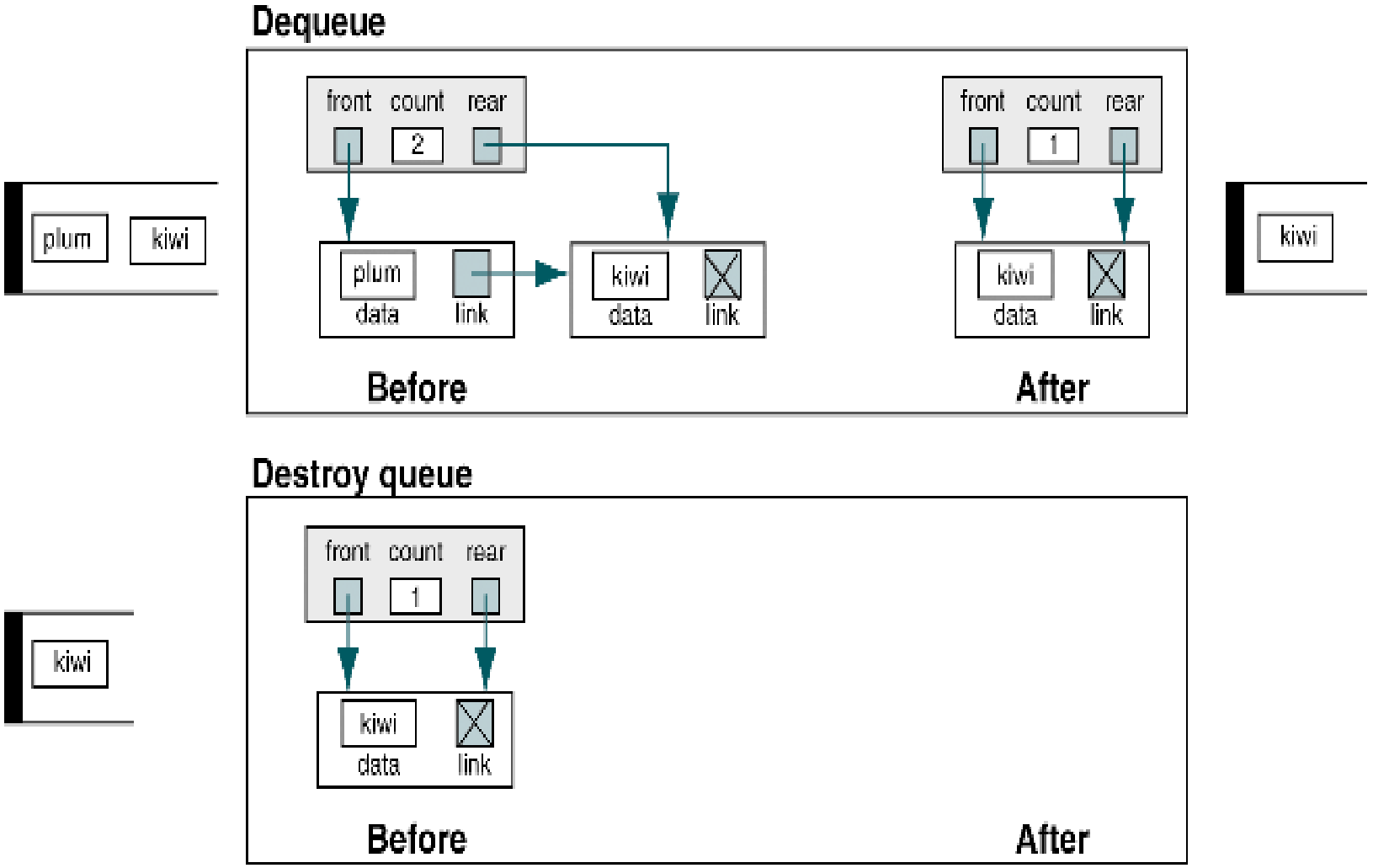
node
  data
  link
end node
```

FIGURE 4-8 Queue Data Structure

Basic Queue Functions

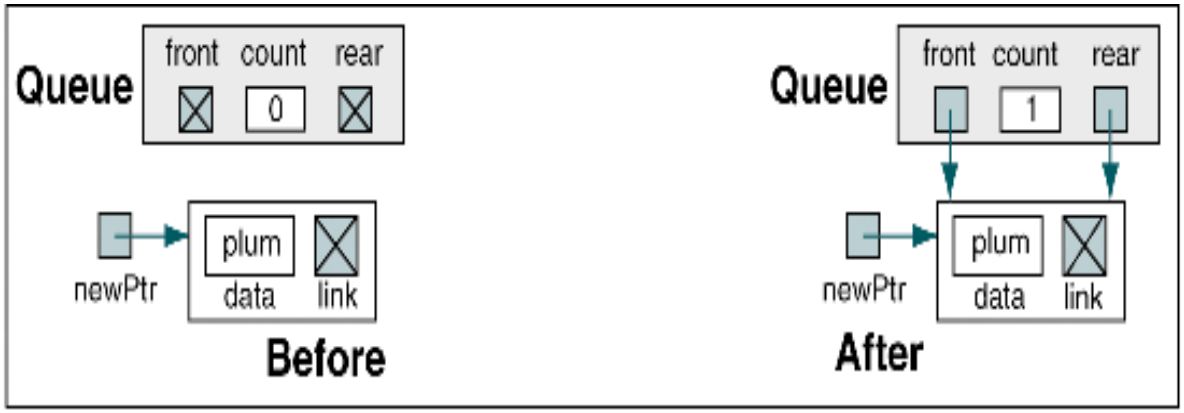


Basic Queue Functions (Continued)

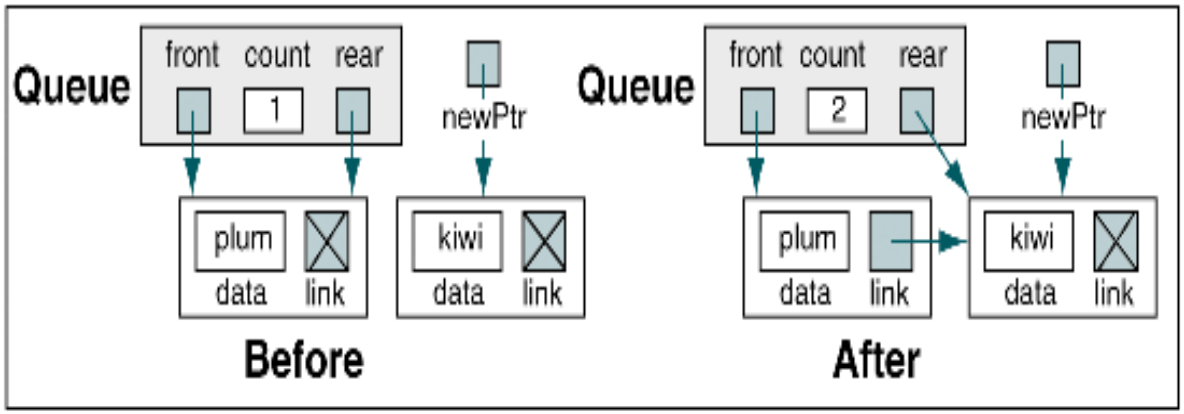


ALGORITHM 4-1 Create Queue

```
Algorithm createQueue
Creates and initializes queue structure.
  Pre    queue is a metadata structure
  Post   metadata elements have been initialized
  Return queue head
1 allocate queue head
2 set queue front to null
3 set queue rear  to null
4 set queue count to 0
5 return queue head
end createQueue
```



(a) Case 1: insert into empty queue

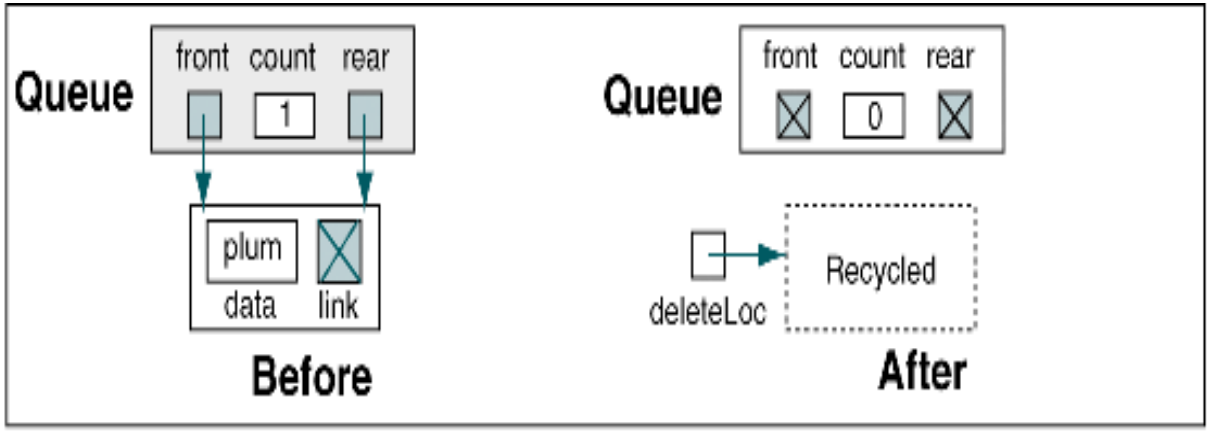


(b) Case 2: insert into queue with data

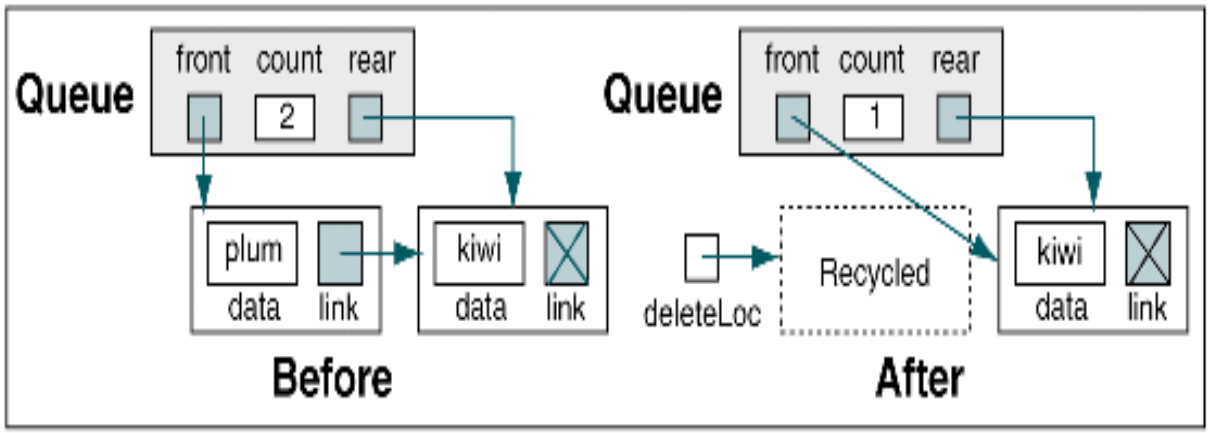
FIGURE 4-10 Enqueue Example

ALGORITHM 4-2 Insert Data into Queue

```
Algorithm enqueue (queue, dataIn)
This algorithm inserts data into a queue.
   Pre    queue is a metadata structure
   Post   dataIn has been inserted
   Return true if successful, false if overflow
 1 if (queue full)
   1  return false
 2 end if
 3 allocate (new node)
 4 move dataIn to new node data
 5 set new node next to null pointer
 6 if (empty queue)
   Inserting into null queue
   1  set queue front to address of new data
 7 else
   Point old rear to new node
   1  set next pointer of rear node to address of new node
 8 end if
 9 set queue rear to address of new node
10 increment queue count
11 return true
end enqueue
```



(a) Case 1: delete only item in queue



(b) Case 2: delete item at front of queue

FIGURE 4-11 Dequeue Examples

ALGORITHM 4-3 Delete Data from Queue

```
Algorithm dequeue (queue, item)
This algorithm deletes a node from a queue.
  Pre    queue is a metadata structure
         item is a reference to calling algorithm variable
  Post   data at queue front returned to user through item
         and front element deleted
  Return true if successful, false if underflow
1 if (queue empty)
  1  return false
2 end if
3 move front data to item
4 if (only 1 node in queue)
  Deleting only item in queue
  1  set queue rear to null
5 end if
6 set queue front to queue front next
7 decrement queue count
8 return true
end dequeue
```

ALGORITHM 4-4 Retrieve Data at Front of Queue

```
Algorithm queueFront (queue, dataOut)
Retrieves data at the front of the queue without changing
queue contents.
  Pre    queue is a metadata structure
         dataOut is a reference to calling algorithm variable
  Post   data passed back to caller
  Return true if successful, false if underflow
1 if (queue empty)
  1  return false
2 end if
3 move data at front of queue to dataOut
4 return true
end queueFront
```

ALGORITHM 4-5 Queue Empty

```
Algorithm emptyQueue (queue)
This algorithm checks to see if a queue is empty.
  Pre    queue is a metadata structure
  Return true if empty, false if queue has data
1 if (queue count equal 0)
  1  return true
2 else
  1  return false
end emptyQueue
```

ALGORITHM 4-6 Full Queue

```
Algorithm fullQueue (queue)
This algorithm checks to see if a queue is full. The queue
is full if memory cannot be allocated for another node.
  Pre    queue is a metadata structure
  Return true if full, false if room for another node
1 if (memory not available)
  1  return true
2 else
  1  return false
3 end if
end fullQueue
```

ALGORITHM 4-7 Queue Count

```
Algorithm queueCount (queue)
This algorithm returns the number of elements in the queue.
  Pre    queue is a metadata structure
  Return queue count
 1 return queue count
end queueCount
```

ALGORITHM 4-8 Destroy Queue

```
Algorithm destroyQueue (queue)
This algorithm deletes all data from a queue.
  Pre     queue is a metadata structure
  Post    all data have been deleted
1 if (queue not empty)
  1  loop (queue not empty)
     1  delete front node
  2  end loop
2 end if
3 delete head structure
end destroyQueue
```

Queue Applications

Categorizing Data

- Consider a list of numbers.
- We need to group the numbers while maintaining their original order in each group.

```
3 22 12 6 10 34 65 29 9 30 81 4 5 19 20 57 44 99
```

We want to categorize them into four different groups:

- Group 1: less than 10
- Group 2: between 10 and 19
- Group 3: between 20 and 29
- Group 4: 30 and greater

```
| 3 6 9 4 5 | 12 10 19 | 22 29 20| 34 65 30 81 57 44 99 |
```

Category Queues

```
Algorithm categorize
  Group a list of numbers into four groups using four queues.
     Written by:
     Date:
 1 createQueue (q0to9)
 2 createQueue (q10to19)
 3 createQueue (q20to29)
 4 createQueue (qOver29)
 5 fillQueues  (q0to9, q10to19, q20to29, qOver29)
 6 printQueues (q0to9, q10to19, q20to29, qOver29)
end categorize
```